

Address and Indirection Operators



Understanding address (&) and indirection (*) operators in C programming

Understanding Address and Indirection Operators

Now we will consider how to determine the address of a variable. The operator that is available in C for this purpose is "&" (address of) operator. The operator & and immediately preceding variable returns the address of variable associated with it.

C's other unary pointer operator is "*", also called as value at address or indirection operator. It returns a value stored at that address. Let us look into illustrative example given below to understand how they are useful.





& Operator

Address of operator

Returns memory address of a variable



* Operator

Value at address operator

Returns value stored at an address

Example Program

Write a program to print address associated with a variable and value stored at that address.



Program to Print Address and Value

```
/*Program to print address associated with a variable and value stored at that address*/  
# include<stdio.h>  
main( )  
{  
    int qty = 5;  
    printf ("Address of qty = %u\n",&qty);  
    printf ("Value of qty = %d \n",qty);  
    printf("Value of qty = %d",*(&qty));  
}
```

Output of Example Program



Program Output

```
Address of qty = 65524  
Value of qty = 5  
Value of qty = 5
```

Explanation



Address Operator (&)

&qty returns the memory address where the variable qty is stored (65524)



Indirection Operator (*)

*(&qty) returns the value stored at the address of qty (5)

Pointer Types Declaration and Assignment

We have seen in previous section that &qty returns address of qty and this address can be stored in a variable as shown below:

```
ptr = &qty;
```

In C, every variable must be declared for its data type before it is used. Even this holds good for pointers too. We know that ptr is not an ordinary variable like any integer variable. We declare data type of pointer variable as that of type of data that will be stored at address to which it is pointing to.

Since ptr is a variable, which contains address of an integer variable qty, it can be declared as:

```
int *ptr;
```

where ptr is called a pointer variable. In C, we define a pointer variable by preceding its name with an asterisk(*). The "*" informs compiler that we want a pointer variable, i.e. to set aside bytes that are required to store address in memory. The int says that we intend to use our pointer variable to store address of an integer.

Example Program with Pointer Declaration



Program Demonstrating Pointer Relationships

```
/* Program below demonstrates relationships we have discussed so far */
#include<stdio.h>
main( )
{
    int qty = 5;
    int *ptr; /* declares ptr as a pointer variable that points to an integer variable */
    ptr = &qty; /* assigning qty's address to ptr -> Pointer Assignment */
    printf ("Address of qty = %u \n", &qty);
    printf ("Address of qty = %u \n", ptr);
    printf ("Address of ptr = %u \n", &ptr);
    printf ("Value of ptr = %d \n", ptr);
    printf ("Value of qty = %d \n", qty);
    printf ("Value of qty = %d \n", *(&qty));
    printf ("Value of qty = %d", *ptr);
}
```

Output of Pointer Example



Program Output

```
Address of qty = 65524  
Address of ptr = 65522  
Value of ptr = 65524  
Value of qty = 5  
Value of qty = 5  
Value of qty = 5
```

Memory Map

Variable qty:

Address: 65524, Value: 5

Pointer ptr:

Address: 65522, Value: 65524

Uninitialized Pointer Example



Program with Uninitialized Pointer

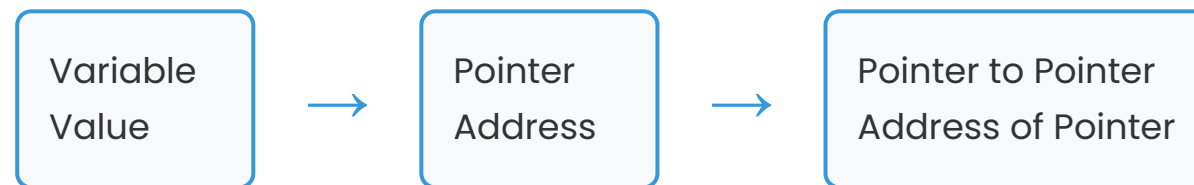
```
/* Program that tries to reference the value of a pointer even though the pointer is uninitialized */
# include<stdio.h>
main()
{
    int *p; /* a pointer to an integer */
    *p = 10;
    printf("the value is %d", *p);
    printf("the value is %u",p);
}
```

Warning

This program demonstrates a common error – using an uninitialized pointer. The pointer 'p' contains an unknown memory address, and assigning a value to *p could overwrite important data or cause a program crash.

Pointer to a Pointer

The concept of pointer can be extended further. As we have seen earlier, a pointer variable can be assigned the address of an ordinary variable. Now, this variable itself could be another pointer. This means that a pointer can contain address of another pointer.



The following program will make the concept clear.

Pointer to Pointer Example



Program with Pointer to Pointer

```
/* Program that declares a pointer to a pointer */
#include<stdio.h>
main( )
{
    int i = 100;
    int *pi;
    int **pii;
    pi = &i;
    pii = &pi;
    printf ("Address of i = %u \n", &i);
    printf ("Address of i = %u \n", pi);
    printf ("Address of i = %u \n", *pii);
    printf ("Address of pi = %u \n", &pi);
}
```

```
printf ("Address of pi = %u \n", pii);  
printf ("Address of pii = %u \n", &pii);  
printf ("Value of i = %d \n", i);  
printf ("Value of i = %d \n", *(&i));  
printf ("Value of i = %d \n", *pi);  
printf ("Value of i = %d", **pii);  
}
```

Output of Pointer to Pointer Example



Program Output

```
Address of i = 65524  
Address of i = 65524  
Address of i = 65524  
Address of pi = 65522  
Address of pi = 65522  
Address of pii = 65520  
Value of i = 100  
Value of i = 100
```

```
Value of i = 100  
Value of i = 100
```

Memory Map

Variable i: Address: 65524, Value: 100

Pointer pi: Address: 65522, Value: 65524

Pointer to pointer pii: Address: 65520, Value: 65522

Null Pointer Assignment

It does make sense to assign an integer value to a pointer variable. An exception is an assignment of 0, which is sometimes used to indicate some special condition. A macro is used

to represent a null pointer. That macro goes under the name NULL.

Thus, setting the value of a pointer using the NULL, as with an assignment statement such as `ptr = NULL`, tells that the pointer has become a null pointer. Similarly, as one can test the condition for an integer value as zero or not, like `if (i == 0)`, as well we can test the condition for a null pointer using `if (ptr == NULL)` or you can even set a pointer to NULL to indicate that it's no longer in use.



NULL Pointer

Pointer with value 0
Indicates special condition



Testing for NULL

`if (ptr == NULL)`
Check if pointer is null



Pointer Usage

Set pointer to NULL
When no longer in use

Null Pointer Example



Program with NULL Pointer

```
#include<stdio.h>
#define NULL 0
main()
{
    int *pi = NULL;
    printf("The value of pi is %u", pi);
}
```



Program Output

The value of pi is 0

Understanding address and indirection operators is fundamental to working with pointers in C.

These operators allow you to access memory directly and create efficient programs.

